

Sparc4D: A Compact Explicit 4D Representation for Dynamic Scenes

Di Yang¹ Yanhai Xiong¹ Yufei Wang²

¹College of William & Mary ²SparcAI Inc.

dyang06@wm.edu yxiong05@wm.edu yufei.wang@sparclab.ai

Abstract

A compact dynamic-scene representation must retain both the surfaces seen over time and the appearance needed to render them from new viewpoints. We present Sparc4D, a feed-forward autoencoder that encodes a monocular video with known cameras into a sparse 4D scene state. Static features are shared across the clip, while spatially anchored temporal slots compress time-varying features. A sparse decoder produces 2D Gaussian surfels, and an optional stored keyframe supplies texture through geometric re-projection. For a 32-frame MultiCamVideo clip, the estimated state averages about 0.4M 32-bit-equivalent values, or 0.6M with one keyframe, including sparse occupancy. With one keyframe, Sparc4D reaches 21.51 dB on the first-32 protocol, compared with 20.40 dB for MoVieS. On randomly placed windows, their PSNR scores are comparable. Diagnostic renderings show that temporal slots preserve target-view PSNR while replacing per-frame latents. Without fine-tuning, Sparc4D also achieves the highest masked PSNR among the four evaluated methods on our windowed DyCheck protocol. These results show that compact, structured 4D states can support competitive feed-forward novel-view reconstruction.

1. Introduction

Feed-forward reconstruction methods can recover dynamic scenes from monocular video and render them from new viewpoints. The resulting scene representation matters as much as the reconstruction procedure: it must retain information across time and remain small enough to store and reuse. Pixel-aligned Gaussian methods such as MoVieS [21] can produce large outputs when motion is materialized across a clip. Compact Gaussian predictors such as C4G [15] reduce the number of primitives, highlighting the tradeoff between representation size and reconstruction fidelity. This raises a representation question: how can a

monocular video be encoded into a compact 4D scene state that supports explicit novel-view rendering?

Video tokenizers [26, 48] compress image sequences, but their pixel-space outputs do not by themselves provide an explicit 3D rendering interface. Sparse 3D autoencoders [41, 42] provide such structure for individual assets. Dynamic scenes require an additional choice: which information should be shared over time, and which should vary? Storing a separate reconstruction for every frame repeats static content. Compressing the entire scene into a coarse latent, on the other hand, makes fine appearance difficult to retain. We address these two requirements separately.

Sparc4D lifts a posed video into sparse 3D observations using estimated depth. Static observations share one latent representation across the clip. Dynamic observations are first encoded per frame, then compressed into a small number of temporal slots at each occupied latent cell. A finer appearance code retains texture separately from the coarse latent. The decoder reconstructs 2D Gaussian surfels [12] for explicit rendering. Sparse occupancy is stored with the features, so it is included in the representation size.

We also allow the state to retain source keyframes. Their pixels are re-projected onto decoded geometry, following image-based rendering and depth-image-based video coding [3, 7, 9, 35]. This gives texture a direct storage path without enlarging every latent cell. Keyframes are optional, and their cost is included in the state. They change rendered colour rather than geometry.

Our experiments examine the resulting size–quality tradeoff on MultiCamVideo [2]. With one keyframe, Sparc4D reaches 21.51 dB on the first-32 protocol and 17.29 dB on randomly placed windows, compared with 20.40 and 17.26 dB for MoVieS. We also compare against reduced-output MoVieS variants and evaluate transfer to real handheld videos on DyCheck [10]. Component analyses examine temporal compression, static and dynamic reconstruction, and the effect of keyframe texture.

The contributions are:

- A sparse 4D scene representation that shares static features and compresses dynamic features into spatially anchored temporal slots.

- A surfel decoder with finer appearance codes and optional stored keyframes, combining learned reconstruction with geometric texture re-projection.
- We show that the proposed state achieves competitive novel-view reconstruction at a substantially smaller representation size than the evaluated feed-forward reconstructors, and transfers from synthetic training data to real videos without fine-tuning.

2. Related Work

Dynamic scene reconstruction. Dynamic radiance fields model time-varying scenes through deformation [28, 29, 31], scene flow [18], or image-based rendering [19]. Gaussian splatting [14] supports deformable and time-varying primitives [25, 40, 46]. Shape of Motion and MoSca regularize motion using low-dimensional bases or scaffolds [17, 38]. These methods optimize a scene-specific representation. We instead learn an encoder and decoder shared across scenes.

Feed-forward reconstruction. Image-based reconstruction networks predict 3D representations in one pass [4, 5, 11, 43, 50]. VGGT and MonST3R estimate geometry and cameras from static or dynamic inputs [36, 49]. For dynamic reconstruction, L4GM and STORM predict time-dependent Gaussians [32, 45]. MoVieS attaches motion to pixel-aligned Gaussians [21], 4DGT predicts space-time Gaussians from monocular video [44], and C4G uses timestamp-conditioned queries to decode a compact Gaussian set [15]. We compare with the latter three using their public weights. Our focus is the encoded scene state and its size–quality tradeoff, rather than the number of output primitives alone.

Scene and video compression. Video tokenizers such as MAGVIT-v2 and Cosmos encode image sequences into spatiotemporal latents [26, 48]. TRELLIS, TRELLIS.2, Sparc3D and XCube use structured or hierarchical sparse representations for 3D assets [20, 33, 41, 42]. SceneTok encodes static scenes into tokens decoded by a diffusion model [1]. We adapt the TRELLIS.2 shape autoencoder to dynamic scenes and add a temporal bottleneck based on learned queries [13, 23]. The resulting state is decoded into surfels and rasterized, without a generative image-refinement or scene-completion module.

Image-based rendering. View-dependent texture mapping, unstructured lumigraphs and learned image-based rendering use reference images to texture scene geometry [3, 7, 34, 37]. Depth-image-based video coding similarly combines stored images and geometry to synthesize

new views [9, 35]. Our optional keyframes follow this principle, but use geometry reconstructed from the stored 4D state rather than transmitted depth maps. For evaluation, we also adopt the distinction between observed and unobserved regions motivated by DyCheck [10].

3. Method

3.1. Scene observations

The input is a monocular clip of $T = 32$ RGB frames $\{I_t\}$ at 256×256 , with known intrinsics and world-to-camera poses. We estimate depth using Depth Anything 3 [22], conditioned on these cameras. A pixel $\mathbf{x}$ is lifted into world coordinates as

$$\mathbf{p}_t(\mathbf{x}) = \mathbf{R}_t^\top (D_t(\mathbf{x})\mathbf{K}^{-1}\tilde{\mathbf{x}} - \mathbf{t}_t), \quad (1)$$

where D_t is estimated depth, $(\mathbf{R}_t, \mathbf{t}_t)$ is the camera pose, and $\tilde{\mathbf{x}}$ is the homogeneous pixel coordinate. Each point carries RGB, a depth-derived normal and its position.

Static and dynamic observations serve different roles in the state. We pool static points across the clip, but retain dynamic points separately for each frame. To separate them, we test whether another frame sees through a point’s expected position. Foreground object masks [52] extend these motion seeds to whole objects, including intervals when an object briefly stops. Appendix A gives the test and mask parameters.

We voxelize the observations inside a scene-adaptive cube at 512^3 resolution. A larger, coarser cube stores static points outside this region. An equirectangular environment map stores more distant observations by ray direction [6]. Pixels covered by neither surfels nor the map receive the source video’s mean colour. Each occupied voxel stores mean RGB, mean normal and a within-voxel point offset. Octant colour residuals retain sub-voxel variation before encoding.

3.2. Shared static features and temporal slots

We initialize the sparse encoder–decoder from the TRELLIS.2 shape autoencoder [42]. It maps a 512^3 sparse voxel grid to a 32-channel latent on a 32^3 grid. Static, coarse-background and per-frame dynamic tensors use the same network weights. The static and coarse latents are stored once per clip. The decoder uses the input occupancy hierarchy for sparse upsampling; this hierarchy is part of the stored state.

The dynamic layer requires time-varying features, but need not store a separate latent for every frame. Let $\mathcal{U}$ be the union of dynamic latent cells over the clip, and $\mathbf{z}_{u,t}$ the latent of cell u at time t . We compress the available per-frame

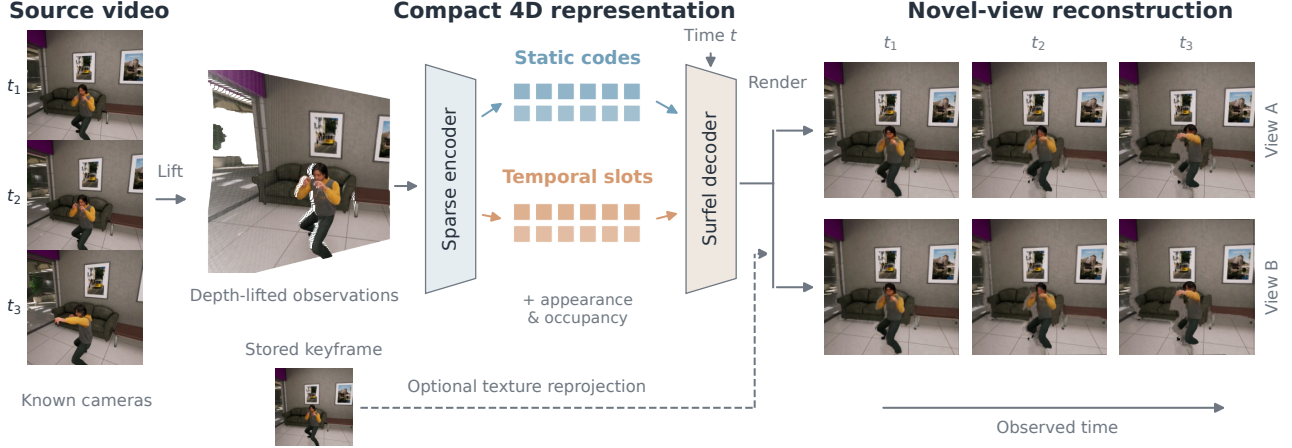


Figure 1. **Overview of Sparc4D.** A posed video is lifted into sparse voxels and encoded into shared static features, dynamic temporal slots and appearance codes. Stored occupancy supports surfel decoding. An optional keyframe supplies re-projected texture. The examples show MultiCamVideo frames 4, 16 and 28; the output rows are Sparc4D+1 KF renderings from two target cameras. The point-cloud inset shows input observations, not decoded geometry. Background components are omitted.

features into two slots per cell:

$$\begin{aligned} \mathbf{k}_{u,t} &= \mathbf{W} \mathbf{z}_{u,t} + \phi_t(t) + \phi_p(u), \\ \mathbf{A}_u &= \text{Attn}(\mathbf{Q} + \phi_p(u), \{\mathbf{k}_{u,t}\}_{t \in \mathcal{T}_u}), \\ \mathbf{S} &= \text{Proj}(\text{Mix}(\{\mathbf{A}_u\}_{u \in \mathcal{U}})), \\ \hat{\mathbf{z}}_{u,t} &= \text{Dec}(\phi_t(t) + \phi_p(u), \mathbf{S}_u). \end{aligned} \quad (2)$$

Here $\mathcal{T}_u$ contains the observed times for cell u , and ϕ_t, ϕ_p embed time and position. Two learned queries $\mathbf{Q}$ first pool features within each cell. Self-attention then mixes the slots across cells, and a projection reduces each slot to 32 channels. To recover a latent, a query at (u, t) attends to the two stored slots of u . The decoded latent $\hat{\mathbf{z}}_{u,t}$ replaces $\mathbf{z}_{u,t}$ in the sparse decoder. The state therefore stores $2|\mathcal{U}|$ latent vectors rather than all per-frame vectors.

3.3. Appearance and surfel decoding

A cell of the 32^3 latent grid spans 16^3 input voxels. We provide a finer path for texture by storing a 16-channel appearance code on the 128^3 grid. For encoder and decoder features $\mathbf{e}_u$ and $\mathbf{g}_u$ at this resolution,

$$\mathbf{a}_u = \mathbf{W}_e \text{LN}(\mathbf{e}_u), \quad \mathbf{g}_u \leftarrow \mathbf{g}_u + \gamma \mathbf{W}_d \mathbf{a}_u, \quad (4)$$

where LN is layer normalization and γ is a fixed gain. Static appearance codes are shared across the clip. Dynamic appearance codes use the temporal-slot construction in Eq. 3, with two 16-channel slots per union cell.

Each decoded voxel emits four 2D Gaussian surfels [12, 30] with degree-1 spherical-harmonics colour. The head predicts centre offsets, scales, rotations and opacity as residuals from a camera-facing base configuration. The base

uses the source camera at the queried time; it does not use the target camera to redefine the surface. We render the surfels with gsplat [47]. Appendix A.1 gives the parameterization. Dynamic decoding uses the occupancy observed at the queried time, so the current model supports novel views at observed timestamps.

3.4. Optional keyframe texture

A small number of stored source images can retain texture without expanding the feature grids. We store k keyframes, evenly spaced within the window; for $k = 1$, we use its first frame. Their pixels contribute to the state size.

For a target camera c at time t , rasterization gives a colour image and median depth $\hat{d}_c$. We back-project a target pixel using $\hat{d}_c$ and project the resulting point into a keyframe. If its projected depth agrees with the depth rendered in that keyframe view, we use the sampled keyframe colour. Otherwise, we retain the surfel colour. Both depth maps come from the decoded state; no target-view depth estimate is needed for this operation.

Static pixels may use any stored keyframe, tried in order of camera proximity [3]. Dynamic pixels use texture only from a keyframe at the same timestamp, because the current state does not store cross-time correspondences. Keyframes change colour only, not rendered geometry or coverage. The projection and consistency test are specified in Appendix A.3.

3.5. Training

For each training sample, we select a 32-frame window and one source camera. Its frames form the input. We sample four timestamps for supervision and render the source view

plus four of the remaining nine cameras at each timestamp. The image loss combines L_1 , SSIM and LPIPS. Depth and normal losses supervise geometry, while a temporal loss matches inter-frame image differences. The temporal slots also receive feature-reconstruction losses. Occupancy and KL losses follow TRELLIS.2 [16, 42].

Training progressively introduces the temporal slots and appearance pathway before joint training on randomly placed windows. The final stage uses one clip per GPU on eight A800 GPUs. The network has 836M trainable parameters, mostly in the pretrained backbone. Appendix B gives the full objective, initialization and training schedule.

3.6. State size

We count the stored features, sparse occupancy, environment map, background colour and optional keyframes. The unit is a *32-bit-equivalent value*: feature values count as one each, while occupancy entropy in bits is divided by 32. For continuity with our measurements, each 8-bit RGB keyframe channel is conservatively charged as one 32-bit value. This is an estimated representation-size ledger, not a measured compressed bitstream. Shared network weights and externally supplied cameras are excluded.

Let $\mathcal{C}$ and $\mathcal{U}$ denote static/coarse cells and dynamic union cells on the 32^3 grid. Their 128^3 counterparts store appearance. With occupancy cost B_{occ} bits, environment texel set $\mathcal{E}$ and k keyframes, the reported size is

$$N = 32|\mathcal{C}| + 64|\mathcal{U}| + 16|\mathcal{C}^{128}| + 32|\mathcal{U}^{128}| + B_{\text{occ}}/32 + 4|\mathcal{E}| + 3 + 3 \cdot 256^2 k. \quad (5)$$

Occupancy is estimated from octree symbols. For dynamic occupancy, we use the smaller estimate from independent frames or consecutive-frame XOR masks. Appendix C details this accounting. Table 1 shows that appearance codes account for most of the stored values.

4. Experiments

We evaluate three questions: the reconstruction quality supported by a compact state, the contribution of temporal compression and keyframe texture, and transfer from synthetic training data to real videos.

4.1. Data and protocol

Dataset and split. MultiCamVideo [2] contains 3,400 Unreal Engine scenes. Each scene is rendered at four focal lengths by ten synchronized cameras for 81 frames at 15 fps. A scene–focal-length pair forms one clip, giving 13,600 clips. Frames are resized to 256^2 .

The held-out test set contains 101 scenes, selected by clustering DINOv2 scene features [27], with all four focal lengths of each scene, giving 404 clips. No test scene is used for training. All MultiCamVideo results in this paper

Table 1. **State size of Sparc4D** in thousands of 32-bit-equivalent values per 32-frame clip, averaged over the test set (random-window protocol; first-32 totals in parentheses). Occupancy cost is estimated from empirical entropy and converted at 32 bits per value. Each keyframe adds $256^2 \times 3$ values.

Component	Mean	Median
Static + coarse latent	14.7	12.1
Dynamic temporal slots	22.7	16.3
Static appearance code	108.4	91.2
Dynamic appearance slots	226.7	154.1
Occupancy, static + coarse	6.3	5.1
Occupancy, dynamic (32 frames)	27.8	19.9
Environment map	10.4	0.0
Total	417.0 (395.8)	324.5 (326.9)
Total with 1 keyframe	613.6 (592.4)	521.1 (523.5)

use this test set. Appendix D.1 gives the construction details.

Windows and cameras. The cameras coincide at the start of each video and then move apart. We therefore evaluate two window placements. The *first-32* protocol uses frames 0–31. The *random-window* protocol fixes one randomly sampled start in $[0, 49]$ for each clip. The source camera is fixed by the clip’s test-list index modulo ten; the other nine cameras are targets. All methods render every target at all 32 timestamps, giving 116,352 target images per protocol.

Metrics. We report image-averaged PSNR, SSIM [39] and AlexNet-LPIPS [51]. Alongside full-image scores, we evaluate pixels classified as observed by a method-independent visibility mask. The mask uses estimated depth and foreground segmentation; dynamic foreground requires same-time source visibility. A third setting, *uniform fill*, assigns the same source-mean colour to all methods outside the mask. These settings separate reconstruction on observed regions from differences in unobserved-region rendering. Appendix D.2 specifies the masks and masked metrics.

Baselines and comparison scope. We evaluate MoVieS [21], 4DGT [44] and C4G [15] with released weights. Sparc4D is trained on the MultiCamVideo training split; this is a comparison against released reconstructors rather than a controlled same-training-data study. All methods render at 256^2 ; input settings and coordinate alignment are given in Appendix D.3. Baseline sizes count their output Gaussian parameters, while our size counts the encoded state.

Table 2. **Novel-view synthesis on the MultiCamVideo test set** (404 clips, 9 target cameras $\times$ 32 frames). Masked metrics use only pixels observed by the source video; uniform fill replaces unobserved pixels of all methods by the same colour. Size is the mean state size in 32-bit-equivalent values (Table 1).

Window	Method	Size	Full image			Observed pixels			Uniform fill		
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	mPSNR $\uparrow$	mSSIM $\uparrow$	mLPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
First-32	4DGT	26.9M	17.18	0.500	0.420	17.57	0.505	0.415	16.70	0.487	0.424
	C4G	0.92M	14.83	0.404	0.535	15.23	0.410	0.528	14.81	0.401	0.518
	MoVieS	228M	20.40	0.643	0.207	21.31	0.658	0.185	19.25	0.618	0.254
	Sparc4D	0.40M	20.06	0.608	0.293	21.09	0.624	0.270	19.14	0.587	0.320
	Sparc4D+1 KF	0.59M	21.51	0.679	0.216	22.78	0.702	0.188	20.21	0.656	0.253
	Sparc4D+4 KF	1.18M	21.57	0.682	0.214	22.85	0.705	0.185	20.28	0.659	0.251
Random	4DGT	26.9M	14.35	0.403	0.550	15.14	0.411	0.538	14.49	0.412	0.546
	C4G	0.92M	13.64	0.369	0.596	14.76	0.394	0.566	14.21	0.393	0.572
	MoVieS	228M	17.26	0.512	0.366	18.55	0.530	0.296	16.43	0.498	0.411
	Sparc4D	0.42M	17.03	0.504	0.421	18.51	0.525	0.350	16.37	0.493	0.444
	Sparc4D+1 KF	0.61M	17.29	0.518	0.379	18.96	0.542	0.300	16.59	0.509	0.409
	Sparc4D+4 KF	1.20M	17.32	0.518	0.378	18.98	0.541	0.299	16.62	0.510	0.409

4.2. Novel-view reconstruction

With one keyframe, Sparc4D reaches 21.51 dB on first-32, compared with 20.40 dB for MoVieS (Table 2), with an estimated state of 0.59M values. On random windows, Sparc4D+1 KF and MoVieS obtain similar PSNR: 17.29 and 17.26 dB. Without keyframes, Sparc4D remains within 0.23–0.34 dB of MoVieS across the two protocols.

The keyframe improves first-32 LPIPS from 0.293 to 0.216 and random-window LPIPS from 0.421 to 0.379. MoVieS retains lower full-image LPIPS at 0.207 and 0.366. Four keyframes add little over one. Section 4.5 examines why the benefit is larger for static content and nearby target views.

On observed pixels, Sparc4D+1 KF reaches 22.78 dB mPSNR on first-32 and 18.96 dB on random windows. The corresponding MoVieS scores are 21.31 and 18.55 dB. Uniform-fill evaluation preserves this PSNR ordering.

Figures 2 and 3 show examples selected near the median PSNR difference to MoVieS. With keyframe texture, static details such as lettering and stone walls become substantially sharper, consistent with the regional analysis in Section 4.5. The selection rule and crop construction are specified in Appendix D.5.

4.3. Size–quality comparison

We evaluate whether reducing the output of a pixel-aligned reconstructor can reach a similar size–quality tradeoff. We give MoVieS fewer input frames or a lower input resolution, or prune its Gaussians by opacity, and evaluate all variants on the first-32 protocol (Table 3).

With four input frames, MoVieS obtains 20.04 dB with 28.5M output values; its single-frame variant obtains 18.97 dB with 7.1M. Sparc4D+1 KF reaches 21.51 dB with

Table 3. **Size and quality** on the first-32 protocol. Size uses 32-bit-equivalent values. MoVieS variants: f input frames at resolution r , or the full model pruned to the k Gaussians with the highest opacity.

Method	Size	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
MoVieS, 32 frames, 224	228M	20.40	0.643	0.207
MoVieS, 8 frames, 224	57.0M	20.28	0.642	0.203
MoVieS, 4 frames, 224	28.5M	20.04	0.628	0.201
MoVieS, 2 frames, 224	14.3M	19.61	0.619	0.219
MoVieS, 1 frame, 224	7.1M	18.97	0.619	0.232
MoVieS, 8 frames, 112	14.3M	16.90	0.435	0.558
MoVieS, top 200k	28.4M	16.25	0.485	0.491
MoVieS, top 50k	7.1M	14.96	0.434	0.642
Sparc4D	0.40M	20.06	0.608	0.293
Sparc4D+1 KF	0.59M	21.51	0.679	0.216
Sparc4D+4 KF	1.18M	21.57	0.682	0.214

0.59M values, about $12\times$ smaller than the single-frame variant. Lowering the input resolution or pruning by opacity reduces quality more strongly at comparable sizes. The reduced-frame variants receive fewer input observations than our 32-frame model.

4.4. Transfer to real videos

We evaluate all four methods without fine-tuning on the seven multi-camera sequences of DyCheck iPhone [10]. The handheld camera provides the input, and the fixed cameras provide target views. Following the windowing used by C4G [15], we take every second frame from non-overlapping 64-timestamp windows to form 32-frame inputs. This yields 40 windows and 1,740 target images. We use the official co-visibility masks with our masked-metric



Figure 2. **Qualitative comparison, first-32 protocol.** Each row shows one test clip at frame 16 from the target camera at median distance to the source camera (distance in metres on the left). Below each image, a $3\times$ enlargement of the red box, chosen automatically as the 64^2 patch with the highest gradient energy in the ground truth. “w/o KF” is Sparc4D without keyframes.

implementation. Windowing, image resampling and metric aggregation differ from the official DyCheck evaluation; the reported numbers are therefore intended for comparison within this table rather than with published DyCheck results. Appendix D.4 gives the preprocessing details.

Sparc4D achieves 14.23 dB mPSNR, compared with 13.95 dB for MoVieS (Table 4). It has higher mPSNR on five of the seven sequences (Table 5), despite training only on synthetic MultiCamVideo. MoVieS achieves lower mLPIPS, 0.488 versus 0.587. Keyframe re-projection improves our mLPIPS to 0.554, while mPSNR decreases to 14.07 dB.

4.5. Representation analysis

Temporal compression. We compare decoded per-frame latents with the temporal slots that replace them, using random windows, eight timestamps per clip and no keyframes (Table 6). Dynamic target-view PSNR is 16.10 dB in both cases, and LPIPS changes from 0.302 to 0.304; source-view PSNR decreases from 22.57 to 22.20 dB. Over all target images of the test set, full-image PSNR is 17.02 dB with per-frame latents and 17.03 dB with slots. Temporal slots therefore provide substantial temporal compression with negligible degradation in target-view reconstruction under this

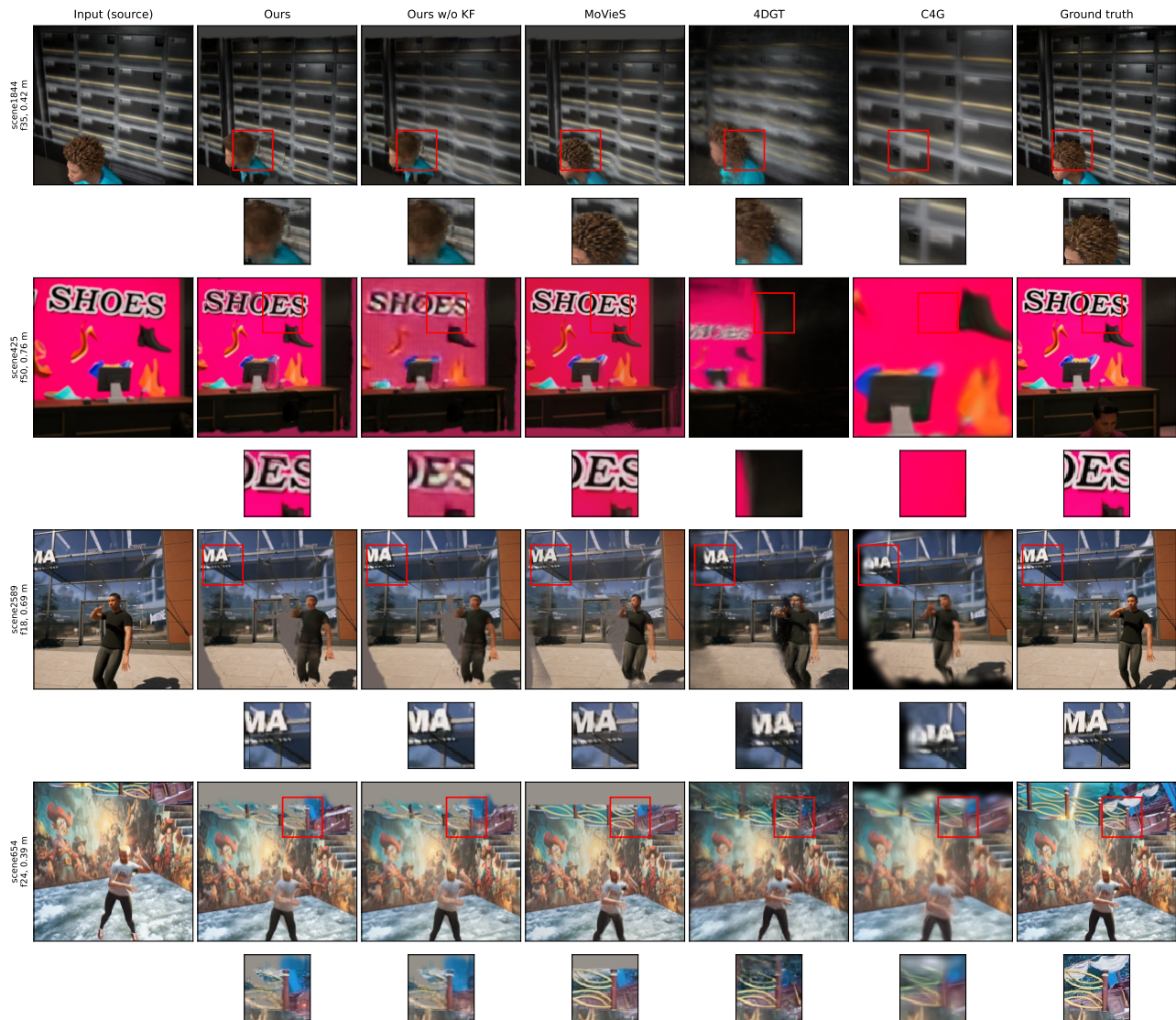


Figure 3. **Qualitative comparison, random-window protocol.** Same layout as Fig. 2; target cameras are restricted to those with at least 60% observed pixels.

setting.

Voxelization and appearance. The first three rows of Table 6 directly render input pixels or voxels as mean-colour disks. They are diagnostic baselines, not reconstruction bounds. At the source view, direct pixel rendering gives 27.27 dB in dynamic regions. Voxelization at 512^3 reduces this to 18.12 dB; 1024^3 recovers 2.0 dB. The learned per-frame decoder reaches 22.57 dB at 512^3 . These results indicate that most of the remaining reconstruction loss arises before temporal slot compression, primarily during voxelized appearance reconstruction.

At target views, Sparc4D exceeds current-frame pixel

rendering by 1.20 dB in dynamic regions, while MoVieS exceeds it by 1.86 dB. Their temporal use differs: MoVieS moves primitives from multiple input frames to a query time, whereas our decoder uses that time’s dynamic occupancy. This structural difference motivates motion-aligned aggregation as a natural extension.

Static and dynamic reconstruction. One keyframe raises static-region PSNR from 22.13 to 25.18 dB on first-32, compared with 21.79 dB for MoVieS (Table 7). The dynamic-region gain is smaller: 18.16 to 18.73 dB. This matches the keyframe rule in Section 3.4: static content can reuse texture across time, but dynamic content borrows tex-

Table 4. **Results on DyCheck iPhone without fine-tuning** under our windowed protocol (7 sequences, 40 windows, 1,740 target images). Our network is trained on synthetic MultiCamVideo only. Masked metrics use the official co-visibility masks.

Method	Co-visible (official masks)			Full image		
	mPSNR $\uparrow$	mSSIM $\uparrow$	mLPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
4DGT	11.84	0.277	0.599	11.37	0.275	0.611
C4G	13.52	0.351	0.634	12.54	0.342	0.651
MoVieS	13.95	0.345	0.488	13.44	0.350	0.517
Sparc4D	14.23	0.350	0.587	13.61	0.353	0.605
Sparc4D+1 KF	14.07	0.328	0.554	13.47	0.334	0.575

Table 5. **mPSNR per DyCheck sequence**, Sparc4D (no keyframe) vs. MoVieS.

Sequence	MoVieS	Sparc4D
apple	15.54	15.98
block	14.84	15.26
paper-windmill	15.79	14.66
space-out	14.16	14.37
spin	13.49	13.70
teddy	11.38	12.72
wheel	12.94	11.78

Table 6. **Rendering ladder** (random windows, 8 times per clip, no keyframes). Rows 1–3 render inputs directly as mean-colour disks and are not upper bounds for the network. “dyn”: dynamic region; “stat”: static region inside the scene cube.

Rendering	Source view, dyn		Target, dyn		Target
	PSNR	LPIPS	PSNR	LPIPS	stat
Current-frame pixels	27.27	0.048	14.90	0.338	14.75
Input voxels, 512 ³	18.12	0.272	14.67	0.419	15.39
Input voxels, dyn. 1024 ³	20.15	0.212	14.81	0.387	15.39
Sparc4D, per-frame latents	22.57	0.137	16.10	0.302	18.55
Sparc4D, temporal slots	22.20	0.143	16.10	0.304	18.55
MoVieS	21.93	0.089	16.76	0.257	18.36

ture only at a stored keyframe’s timestamp.

All methods use the same dynamic-region mask, obtained by projecting the current input’s dynamic voxels into the target view. Dynamic LPIPS is computed on the region’s bounding box.

Keyframe benefit over time. Table 8 reports sampled timestamps from random windows over the test set. One keyframe improves PSNR by 1.4 dB at $t = 0$ and 0.1 dB at $t = 31$. The gain also declines when all input frames are stored as keyframes, so temporal distance alone does not explain the trend. The decline is consistent with increasing source–target separation, which reduces visibility overlap and makes re-projection more sensitive to depth errors. At every sampled timestamp, one keyframe improves PSNR

Table 7. **Static and dynamic regions**, first-32 protocol. Dynamic-region LPIPS is computed on the bounding box of the region.

Method	Dynamic		Static
	PSNR $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$
4DGT	16.19	0.345	18.56
C4G	14.32	0.434	15.97
MoVieS	19.02	0.150	21.79
Sparc4D	18.16	0.219	22.13
Sparc4D+1 KF	18.73	0.180	25.18
Sparc4D+4 KF	18.78	0.176	25.27

Table 8. **PSNR by frame index within the window** (random windows, mean over 9 target cameras, computed from 8-bit renders).

t	0	4	8	16	24	31
Sparc4D	18.13	17.77	17.43	16.89	16.52	16.23
Sparc4D+1 KF	19.49	18.23	17.75	17.07	16.64	16.31
Sparc4D+32 KF	19.42	18.39	17.84	17.11	16.65	16.30
MoVieS	18.18	17.93	17.67	17.17	16.81	16.50

over the model without keyframes.

Appendix E provides an additional analysis of decoded depth consistency, where Sparc4D more closely preserves the DA3 depth structure than the evaluated baselines.

5. Limitations

The current decoder uses per-frame dynamic occupancy and renders at observed timestamps; it does not transport dynamic surfaces across time, so content hidden from the source camera at a query time is not recovered from other frames. Fine detail on moving content is also less well preserved than static texture.

6. Conclusion

Sparc4D encodes a monocular video into a compact sparse 4D state. Shared static features and temporal slots reduce repeated storage, while finer appearance codes and optional keyframes retain texture. The state decodes directly into explicit surfels for novel-view rendering. Experiments demonstrate a favorable size–quality tradeoff against the evaluated feed-forward reconstructors and output-reduction strategies, together with transfer from synthetic training data to real videos. These results support sparse, temporally compressed scene states as a practical representation for feed-forward dynamic-scene reconstruction.

References

- [1] Mohammad Asim, Christopher Wewer, and Jan Eric Lenssen. SceneTok: A compressed, diffusable token space for 3D scenes. In *Proceedings of the IEEE/CVF Conference*

- on *Computer Vision and Pattern Recognition (CVPR)*, 2026. 2
- [2] Jianhong Bai, Menghan Xia, Xiao Fu, Xintao Wang, Lianrui Mu, Jinwen Cao, Zuozhu Liu, Haoji Hu, Xiang Bai, Pengfei Wan, and Di Zhang. ReCamMaster: Camera-controlled generative rendering from a single video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. 1, 4
 - [3] Chris Buehler, Michael Bosse, Leonard McMillan, Steven Gortler, and Michael Cohen. Unstructured lumigraph rendering. In *Proceedings of SIGGRAPH*, pages 425–432, 2001. 1, 2, 3
 - [4] David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelSplat: 3D Gaussian splats from image pairs for scalable generalizable 3D reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2
 - [5] Yuedong Chen, Haoifei Xu, Chuanxia Zheng, Bohan Zhuang, Marc Pollefeys, Andreas Geiger, Tat-Jen Cham, and Jianfei Cai. MVSplat: Efficient 3D Gaussian splatting from sparse multi-view images. In *European Conference on Computer Vision (ECCV)*, 2024. 2
 - [6] Paul Debevec. Rendering synthetic objects into real scenes: Bridging traditional and image-based graphics with global illumination and high dynamic range photography. In *Proceedings of SIGGRAPH*, pages 189–198, 1998. 2
 - [7] Paul E. Debevec, Camillo J. Taylor, and Jitendra Malik. Modeling and rendering architecture from photographs: A hybrid geometry- and image-based approach. In *Proceedings of SIGGRAPH*, pages 11–20, 1996. 1, 2
 - [8] David Eigen, Christian Puhersch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. 14
 - [9] Christoph Fehn. Depth-image-based rendering (DIBR), compression, and transmission for a new approach on 3D-TV. In *Stereoscopic Displays and Virtual Reality Systems XI*, pages 93–104, 2004. 1, 2
 - [10] Hang Gao, Ruilong Li, Shubham Tulsiani, Bryan Russell, and Angjoo Kanazawa. Monocular dynamic view synthesis: A reality check. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 1, 2, 5
 - [11] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. LRM: Large reconstruction model for single image to 3D. In *International Conference on Learning Representations (ICLR)*, 2024. 2
 - [12] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D Gaussian splatting for geometrically accurate radiance fields. In *SIGGRAPH 2024 Conference Papers*, 2024. 1, 3, 12, 13
 - [13] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning (ICML)*, 2021. 2
 - [14] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. 2
 - [15] Mungyeom Kim, Minkyong Jeon, Honggyu An, Jaewoo Jung, Hyuna Ko, Jisang Han, Hyeonseo Yu, Donghwan Shin, Sunghwan Hong, Takuya Narihira, et al. Learning global motion with compact Gaussians for feed-forward 4D reconstruction. *arXiv preprint arXiv:2605.31595*, 2026. 1, 2, 4, 5
 - [16] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014. 4, 13
 - [17] Jiahui Lei, Yijia Weng, Adam W. Harley, Leonidas Guibas, and Kostas Daniilidis. MoSca: Dynamic Gaussian fusion from casual videos via 4D motion scaffolds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. 2
 - [18] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2
 - [19] Zhengqi Li, Qianqian Wang, Forrester Cole, Richard Tucker, and Noah Snavely. DynIBaR: Neural dynamic image-based rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2
 - [20] Zhihao Li, Yufei Wang, Heliang Zheng, Yihao Luo, and Bihan Wen. Sparc3d: Sparse representation and construction for high-resolution 3d shapes modeling. *Advances in Neural Information Processing Systems*, 38:118582–118600, 2026. 2
 - [21] Chenguo Lin, Yuchen Lin, Panwang Pan, Yifan Yu, Tao Hu, Honglei Yan, Katerina Fragkiadaki, and Yadong Mu. MoVieS: Motion-aware 4D dynamic view synthesis in one second. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. 1, 2, 4
 - [22] Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025. 2
 - [23] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 2
 - [24] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. 13
 - [25] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3D Gaussians: Tracking by persistent dynamic view synthesis. In *International Conference on 3D Vision (3DV)*, 2024. 2
 - [26] NVIDIA. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025. 1, 2
 - [27] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al.

- DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 4
- [28] Keunhong Park, Utkarsh Sinha, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Steven M. Seitz, and Ricardo Martin-Brualla. Nerfies: Deformable neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. 2
- [29] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Ricardo Martin-Brualla, and Steven M. Seitz. HyperNeRF: A higher-dimensional representation for topologically varying neural radiance fields. *ACM Transactions on Graphics*, 40(6), 2021. 2
- [30] Hanspeter Pfister, Matthias Zwicker, Jeroen van Baar, and Markus Gross. Surfels: Surface elements as rendering primitives. In *Proceedings of SIGGRAPH*, pages 335–342, 2000. 3, 12
- [31] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2
- [32] Jiawei Ren, Kevin Xie, Ashkan Mirzaei, Hanxue Liang, Xiaohui Zeng, Karsten Kreis, Ziwei Liu, Antonio Torralba, Sanja Fidler, Seung Wook Kim, and Huan Ling. L4GM: Large 4D Gaussian reconstruction model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 2
- [33] Xuanchi Ren, Jiahui Huang, Xiaohui Zeng, Ken Museth, Sanja Fidler, and Francis Williams. XCube: Large-scale 3D generative modeling using sparse voxel hierarchies. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2
- [34] Gernot Riegler and Vladlen Koltun. Free view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020. 2
- [35] Gerhard Tech, Ying Chen, Karsten Müller, Jens-Rainer Ohm, Anthony Vetro, and Ye-Kui Wang. Overview of the multiview and 3D extensions of high efficiency video coding. *IEEE Transactions on Circuits and Systems for Video Technology*, 26(1):35–49, 2016. 1, 2
- [36] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. 2
- [37] Qianqian Wang, Zhicheng Wang, Kyle Genova, Pratul P. Srinivasan, Howard Zhou, Jonathan T. Barron, Ricardo Martin-Brualla, Noah Snavely, and Thomas Funkhouser. IBRNet: Learning multi-view image-based rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. 2
- [38] Qianqian Wang, Vickie Ye, Hang Gao, Jake Austin, Zhengqi Li, and Angjoo Kanazawa. Shape of motion: 4D reconstruction from a single video. *arXiv preprint arXiv:2407.13764*, 2024. 2
- [39] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. 4
- [40] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2
- [41] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3D latents for scalable and versatile 3D generation. *arXiv preprint arXiv:2412.01506*, 2024. 1, 2
- [42] Jianfeng Xiang, Xiaoxue Chen, Sicheng Xu, Ruicheng Wang, Zelong Lv, Yu Deng, Hongyuan Zhu, Yue Dong, Hao Zhao, Nicholas Jing Yuan, and Jiaolong Yang. Native and compact structured latents for 3D generation. *Tech report*, 2025. 1, 2, 4, 13
- [43] Haofei Xu, Songyou Peng, Fangjinhua Wang, Hermann Blum, Daniel Barath, Andreas Geiger, and Marc Pollefeys. DepthSplat: Connecting Gaussian splatting and depth. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. 2
- [44] Zhen Xu, Zhengqin Li, Zhao Dong, Xiaowei Zhou, Richard Newcombe, and Zhaoyang Lv. 4DGT: Learning a 4D Gaussian transformer using real-world monocular videos. *arXiv preprint arXiv:2506.08015*, 2025. 2, 4
- [45] Jiawei Yang, Jiahui Huang, Yuxiao Chen, Yan Wang, Boyi Li, Yurong You, Apoorva Sharma, Maximilian Igl, Peter Karkus, Danfei Xu, Boris Ivanovic, Yue Wang, and Marco Pavone. STORM: Spatio-temporal reconstruction model for large-scale outdoor scenes. In *International Conference on Learning Representations (ICLR)*, 2025. 2
- [46] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3D Gaussians for high-fidelity monocular dynamic scene reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2
- [47] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for Gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025. 3, 12
- [48] Lijun Yu, José Lezama, Nitesh B. Gundavarapu, Luca Versari, Kihyuk Sohn, David Minnen, Yong Cheng, Agrim Gupta, Xiuye Gu, Alexander G. Hauptmann, Boqing Gong, Ming-Hsuan Yang, Irfan Essa, David A. Ross, and Lu Jiang. Language model beats diffusion – tokenizer is key to visual generation. In *International Conference on Learning Representations (ICLR)*, 2024. 1, 2
- [49] Junyi Zhang, Charles Herrmann, Junhwa Hur, Varun Jampani, Trevor Darrell, Forrester Cole, Deqing Sun, and Ming-Hsuan Yang. MonST3R: A simple approach for estimating geometry in the presence of motion. In *International Conference on Learning Representations (ICLR)*, 2025. 2
- [50] Kai Zhang, Sai Bi, Hao Tan, Yuanbo Xiangli, Nanxuan Zhao, Kalyan Sunkavalli, and Zexiang Xu. GS-LRM: Large reconstruction model for 3D Gaussian splatting. In *European Conference on Computer Vision (ECCV)*, 2024. 2
- [51] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of

deep features as a perceptual metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. [4](#)

- [52] Peng Zheng, Dehong Gao, Deng-Ping Fan, Li Liu, Jorma Laaksonen, Wanli Ouyang, and Nicu Sebe. Bilateral reference for high-resolution dichotomous image segmentation. *CAAI Artificial Intelligence Research*, 3, 2024. [2](#), [12](#)

A. Observation preprocessing and decoding

Depth and points. We run pose-conditioned Depth Anything 3 on overlapping 32-frame chunks of the full video. Consecutive chunks are aligned in scale using the median depth ratio on their overlap. Equation 1 maps pixels to world coordinates. Normals are computed from depth gradients, and sky pixels flagged by the depth model are excluded from the point cloud.

Motion seeds. Let $(\mathbf{x}', z')$ be the projection of $\mathbf{p}_t(\mathbf{x})$ into another frame t' . The point is marked dynamic if, for some $|t - t'| \in \{1, 2, 4, 8, 16\}$,

$$\frac{D_{t'}(\mathbf{x}') - z'}{\max(z', 0.05 \text{ m})} > 0.08. \quad (6)$$

We set $D_{t'} = \infty$ on sky. The test detects a later or earlier view seeing through the expected location of a point. Connected components smaller than 48 pixels are removed, and the mask is dilated by four pixels.

Object masks. RMBG-2.0 (BiRefNet [52]) provides foreground components. An 8-connected component c is selected if it contains at least $\max(16, 0.02|c|)$ motion-seed pixels and covers at most 35% of the image. Selections propagate forward and backward to components overlapping a selected region by at least 30% of their area. This keeps an object marked during brief pauses. Static points are pooled across all 32 frames; dynamic points remain per-frame.

Spatial layout. The scene cube has side length $s = \text{clip}(3\tilde{d}, 4 \text{ m}, 32 \text{ m})$, where $\tilde{d}$ is median camera-to-point distance. Its centre is the median of the nearest 30% of points. A 512^3 grid gives voxel widths of approximately 0.8–6.3 cm. A concentric 64 m cube at the same grid resolution stores outlying static points with 12.5 cm voxels. Points beyond that cube contribute to a 1024×512 equirect-angular map by viewing direction relative to the camera-trajectory centre. Only sampled texels are stored. The map is rendered behind the surfels; remaining pixels receive the video’s mean RGB.

Voxel features. Each voxel has nine base features: mean RGB, mean normal and mean within-voxel point offset. For each of eight octants we append the difference between its mean RGB and the voxel mean, plus an occupancy flag. These 32 additional features retain colour variation within a voxel.

A.1. Surfel parameterization

Each output voxel $\mathbf{v}$ (integer coordinates, voxel size h , cube origin $\mathbf{o}$) emits $K=4$ 2D Gaussian surfels [12, 30] with

degree-1 spherical-harmonics colour. The head predicts residuals $(\delta, \ell, \hat{\mathbf{q}}, a)$ around a base configuration that depends on the source camera centre $\mathbf{c}_t$ at the rendered time t :

$$\begin{aligned} \boldsymbol{\mu} &= \mathbf{o} + h \left(\mathbf{v} + \frac{1}{2} + \tanh \delta \right), & \alpha &= \sigma(a + 2), \\ \mathbf{s} &= \text{cap} \left(\rho \frac{\text{softplus}(\ell + b)}{\text{softplus}(b)} \right), & \rho &= \frac{0.55 \|\mathbf{c}_t - \bar{\mathbf{v}}\|}{f}, \\ \mathbf{q} &= \text{norm}(\hat{\mathbf{q}} + \mathbf{1}_w) \otimes \mathbf{q}_{\text{face}}(\mathbf{c}_t - \bar{\mathbf{v}}), \end{aligned} \quad (7)$$

where $\bar{\mathbf{v}}$ is the voxel centre, ρ is the world-space footprint of 0.55 pixels at focal length f , $b = \log(e - 1)$, $\text{cap}(s) = s s_{\text{max}} / (s + s_{\text{max}})$ with s_{max} four voxels, $\mathbf{1}_w$ the identity quaternion, and $\mathbf{q}_{\text{face}}(\mathbf{n})$ the rotation that turns the surfel normal towards $\mathbf{n}$. With a zero-initialized output layer the head therefore renders each voxel as a camera-facing disk of about one pixel, providing the initial configuration for residual prediction. Coarse-cube voxels use the same head with their own voxel size and a minimum footprint of half a coarse voxel. Surfels are rasterized with gsplat [47].

A.2. Appearance and slot modules

In Eq. 4, the encoder projection $\mathbf{W}_e$ uses its default initialization. The decoder projection $\mathbf{W}_d$ is zero-initialized. The octant-feature input projection is also zero-initialized. Both residual pathways use gain $\gamma = 10$, so their initial contribution is zero. Gradients in the gained layers are scaled by $1/\gamma^2$ before clipping.

The temporal modules use width 256 and eight attention heads. Two learned queries pool the observed tokens of each cell. Three self-attention layers mix all slots, followed by projection to the stored channel dimension. Three cross-attention layers decode each (u, t) query. Latent slots have 32 channels; appearance slots have 16. Both receive the normalized feature-matching losses in Appendix B.

A.3. Keyframe re-projection

For $k > 1$, keyframes are selected at $\mathcal{K} = \{\text{round}(31i/(k-1))\}_{i=0}^{k-1}$. For $k = 1$, $\mathcal{K} = \{0\}$. Images are stored at 256^2 .

Given target depth $\hat{d}_c$, a pixel is back-projected to $\mathbf{X} = \pi_c^{-1}(\mathbf{x}, \hat{d}_c(\mathbf{x}))$ and projected into keyframe τ as $(\mathbf{y}, z) = \pi_\tau(\mathbf{X})$. Here π_c is the calibrated camera projection. We accept the sampled colour $I_\tau(\mathbf{y})$ when

$$|z - \hat{d}_\tau(\mathbf{y})| < 0.03z + 0.02 \text{ m}, \quad (8)$$

where $\hat{d}_\tau$ is rendered from the same decoded state in the keyframe view. Otherwise the surfel colour is retained. Static pixels try keyframes in order of source-camera distance to the target camera. Pixels where the dynamic layer alone has opacity above 0.5 use only same-time keyframe colour. Keyframe colours are re-projected per pixel.

B. Training details

For each training sample, we select a 32-frame window and one source camera. Its frames form the input. We sample four timestamps for supervision. At each timestamp, the model renders the source view and four randomly selected target views from the remaining nine cameras. The loss is

$$\mathcal{L} = \frac{1}{4} \sum_q \mathcal{L}_q + 0.5 \mathcal{L}_{\Delta t} + \mathcal{L}_{\text{lat}} + \mathcal{L}_{\text{lat}}^{\text{app}} + 0.1 \mathcal{L}_{\text{occ}} + 10^{-6} \mathcal{L}_{\text{KL}}, \quad (9)$$

$$\mathcal{L}_q = \mathcal{L}_{\text{rgb}}^{\text{src}} + \frac{1}{4} \sum_v \mathcal{L}_{\text{rgb}}^v + 0.3 \left(\mathcal{L}_d^{\text{src}} + \frac{1}{4} \sum_v \mathcal{L}_d^v \right) + 0.05 \mathcal{L}_n, \quad (10)$$

where v runs over the four supervising target cameras. $\mathcal{L}_{\text{rgb}} = \|\hat{I} - I\|_1 + 0.2(1 - \text{SSIM}) + 0.6 \text{LPIPS}_{\text{VGG}}$ on the image composited over the mean-colour background. $\mathcal{L}_d$ is an L_1 loss on log depth against the per-camera depth estimate, restricted to pixels inside the scene cube and covered by surfels. $\mathcal{L}_n$ is the 2DGS normal-consistency term [12]. $\mathcal{L}_{\Delta t} = \text{mean}_{a < b} \|(\hat{I}_a - \hat{I}_b) - (I_a - I_b)\|_1$ penalizes temporal differences that do not match the ground truth. $\mathcal{L}_{\text{lat}}$ and $\mathcal{L}_{\text{lat}}^{\text{app}}$ are the relative squared errors between slot-decoded and per-frame latents and appearance codes (for the appearance code on a random subset of 65,536 rows). $\mathcal{L}_{\text{occ}}$ and $\mathcal{L}_{\text{KL}}$ are the occupancy and KL terms of TRELLIS.2 [16, 42].

Training proceeds in four stages, each warm-started from the previous one.

Stage 1. The autoencoder and surfel head are trained for 36k steps on the first 32 frames of the training clips.

Stage 2. Temporal slots are added and trained for 12k steps with the encoder frozen.

Stage 3. Octant features and the appearance code are added, Stage 3 introduces the see-through motion mask, and the model is trained for 8k steps.

Stage 4. The model is trained for 30k steps on randomly placed 32-frame windows of the full 81-frame clips, with the object-level motion masks, the slot-compressed dynamic appearance code and the LPIPS weight is 0.2 in Stages 1–3 and 0.6 in Stage 4..

Optimization. Stage 4 uses AdamW [24] ($\beta = (0.9, 0.95)$, no weight decay), learning rate 3×10^{-5} with 200 warm-up steps and cosine decay to 10^{-5} , a $10\times$ learning rate for the slot modules, and one clip per GPU on 8 A800 GPUs (about 33 hours). The model has 836M trainable parameters, almost all in the TRELLIS.2 backbone.

C. Representation-size accounting

Equation 5 counts static/coarse latents, dynamic latent slots, static/coarse appearance codes, dynamic appearance slots, occupancy, environment texels, mean background RGB and optional keyframes. Occupancy uses a 32^3 bitmap followed by 8-bit child masks at each of four finer octree levels. We estimate its cost using the zeroth-order empirical entropy of the symbols. For dynamic occupancy we take the smaller of independent per-frame coding and consecutive-frame XOR-mask coding. The estimate is not the length of an implemented arithmetic-coded bitstream.

We report these terms in 32-bit-equivalent values. Each stored feature scalar counts as one value and occupancy contributes $B_{\text{occ}}/32$. Each environment texel counts as four values. An RGB keyframe contributes $3 \cdot 256^2$ values even though its channels are stored as bytes. This conservative keyframe charge is retained across all tables. The ledger is not a measurement of serialized file size or peak rendering memory. It does not include shared model weights or externally supplied cameras. The displayed formula also does not itemize scene-layout metadata such as cube origin and scale. Table 1 reports means and medians over the test set.

For MoViS, the materialized output contains 1,605,632 Gaussians at 32×224^2 input pixels. We count 14 static parameters and 4×32 query-time motion parameters per Gaussian, or 142 values. For 4DGT we count the predicted Gaussian parameters. For C4G we count the outputs of 2,048 Gaussians at each of the 32 queried timestamps. These are output-size comparisons, not comparisons of identically compressed latent caches.

D. Evaluation details

D.1. Split and camera separation

We compute a DINOv2 environment feature for each scene and cluster the 3,400 scenes with k -means ($k = 120$). Test scenes are drawn from clusters assigned to the test side, and each has cosine similarity below 0.62 to every scene in the remaining clusters. This gives 101 test scenes, each rendered at four focal lengths (404 clips). Training stages 1–3 use 3,170 scenes (12,680 clips); stage 4 uses all 3,299 non-test scenes (13,196 clips). No test scene appears in any training stage.

The four focal lengths are 18, 24, 35 and 50 mm. The ten cameras coincide at frame 0 and separate over time. Measured over 404 test clips, mean pairwise separation is 0.93 m at frame 31 and 2.10 m at frame 80. In first-32, 70% of target images are within 0.5 m of the source camera; in random windows, 35% are. The two protocols therefore cover different view-separation regimes.

D.2. Visibility masks and image metrics

MultiCamVideo provides no co-visibility masks. We construct a shared mask from estimated depth and foreground segmentation, independently of each reconstruction method. A target pixel is back-projected using its estimated depth. Depth scale is aligned to the source using a median depth ratio before projecting the point into source frames.

A background pixel is accepted if at least one source frame sees a background surface at the projected position with depth agreement within 10%. An RGB foreground pixel is accepted only through the source frame at the same time. This restriction avoids treating a moving point as static across frames, but does not identify foreground observations connected by motion. Sky and points beyond 40 m are checked by direction. The resulting masks cover on average 83% of target pixels for first-32 and 64% for random windows.

For mask M and RGB intensities in $[0, 1]$, we use

$$\text{mPSNR} = -10 \log_{10} \frac{\sum_{\mathbf{x}} M(\mathbf{x}) \|\hat{I}(\mathbf{x}) - I(\mathbf{x})\|^2}{3 \sum_{\mathbf{x}} M(\mathbf{x})}. \quad (11)$$

mSSIM and mLPIPS average their spatial metric maps inside the mask. Images with less than 1% observed pixels are omitted from masked averages, about 2% of random-window images. Masked and uniform-fill scores use stored 8-bit renderings. Uniform fill replaces pixels outside M by the source video’s mean colour before computing full-image metrics.

For region analysis, current-frame dynamic input voxels are rendered as camera-facing disks into each target camera. We threshold opacity at 0.5 and dilate by two pixels. The static region lies inside the scene cube. Dynamic-region LPIPS uses the region’s bounding box, padded by eight pixels and resized to 128^2 , rather than a masked full-image LPIPS map.

D.3. Baseline configurations

MoVieS receives all 32 frames at 224^2 , together with ground-truth cameras and timestamps. 4DGT receives all 32 frames at 560^2 and ground-truth cameras. Its timestamps use frame index divided by five rather than the dataset’s physical rate of 15 fps. We select this configuration using source-view reconstruction quality only, without access to target-view ground-truth metrics. We report the selection criterion without treating this as a matched-training comparison.

C4G is pose-free and receives 12 frames spread uniformly over the window at 224^2 , matching its training context count. We align its arbitrary reconstruction scale to the estimated depth of its first context frame using a median ratio. Target cameras are expressed relative to that frame. All methods render at 256^2 and composite over the source mean colour. The output-size accounting is given in Appendix C.

D.4. DyCheck preprocessing

We use the *apple*, *block*, *paper-windmill*, *space-out*, *spin*, *teddy* and *wheel* sequences. The handheld video supplies 32 input frames from every second timestamp of a non-overlapping 64-timestamp window. At 30 fps this gives the 15 fps input sampling used in training. Targets are fixed-camera images present in the official validation split at those input times. Windows with no target image are dropped.

We centre-crop the half-resolution 360×480 images to 360^2 and resize to 256^2 . Images are resampled to the intrinsics of the first handheld frame in each window. For a pinhole camera, changing intrinsics while keeping pose fixed induces the corresponding image-plane transform. Official co-visibility masks use nearest-neighbour resampling. Source depth and foreground masks are prepared as for MultiCamVideo. The baselines use Appendix D.3’ settings; all receive known cameras except pose-free C4G.

DyCheck supplies LiDAR depth for the input camera, not the fixed evaluation cameras; no target-view LiDAR comparison is reported.

D.5. Qualitative selection

For each protocol, examples are selected near the median per-clip PSNR difference between Sparc4D+1 KF and MoVieS, using different environments. Random-window examples additionally require at least 60% observed target pixels. The figures show timestamp 16 and a target camera at median distance from the source. The enlarged 64^2 patch is selected by maximum gradient energy in the ground-truth image.

E. Depth consistency

MultiCamVideo does not provide ground-truth depth. We compare rendered target-view depth with per-camera DA3 estimates using AbsRel, RMSE and δ_1 , the fraction of pixels whose depth ratio is within 1.25 [8]. We also report median-scaled AbsRel and δ_1 . Each method is evaluated where it renders a surface and the reference is valid; coverage is reported alongside the errors.

Rendered depth from Sparc4D agrees more closely with the DA3 reference in this evaluation. After median scaling, AbsRel is 0.111 on first-32 and 0.164 on random windows, compared with 0.250 and 0.272 for MoVieS. This measures consistency with DA3 rather than independent geometric accuracy, since DA3 also provides our input depth and training supervision. Median alignment removes global scale differences but does not eliminate shared DA3 bias or differences in valid rendered regions.

Keyframes affect colour only and have the same rendered depth as the base model.

Table 9. **Rendered depth vs. the per-camera depth estimate** at target views (test set, per-image average). “aligned”: per-image median scaling.

	Method	AbsRel↓	$\delta_1 \uparrow$	RMSE↓	aligned		cov.
					AbsRel↓	$\delta_1 \uparrow$	
First-32	4DGT	0.624	0.141	8.56	0.344	0.621	0.92
	C4G	0.505	0.538	6.34	0.476	0.586	0.91
	MoVieS	0.561	0.513	6.21	0.250	0.717	0.92
	Sparc4D	0.163	0.859	1.98	0.111	0.896	0.87
Random	4DGT	0.640	0.155	8.40	0.370	0.598	0.81
	C4G	0.543	0.522	6.12	0.450	0.596	0.84
	MoVieS	0.615	0.505	5.84	0.272	0.704	0.75
	Sparc4D	0.254	0.792	2.36	0.164	0.840	0.72